

txt2tex DAT Reference

Database Design: primary keys, relational algebra, bindings, GROUP/UNGROUP.

<https://github.com/jmf-pobox/txt2tex>

Document Structure

=== Title ===	<code>\section *{Title}</code>
* Solution N **	Bold heading with spacing
(a) Text	Part label
TEXT: paragraph	Prose block (smart formula detection)
LATEX: \cmd	Raw L ^A T _E X passthrough
PAGEBREAK:	Page break
LINEBREAK:	Vertical gap (<code>\medskip</code>) — between blocks, no page break
TITLE: ...	Document title
AUTHOR: ...	Author name
DATE: ...	Date string
BIBLIOGRAPHY: f.bib	BibTeX file
[cite key]	<code>\citep {key}</code>

Logic

<code>p land q</code>	$p \wedge q$
<code>p lor q</code>	$p \vee q$
<code>lnot p</code>	$\neg p$
<code>p => q</code>	$p \Rightarrow q$
<code>p <=> q</code>	$p \Leftrightarrow q$
<code>forall x : S P</code>	$\forall x : S \bullet P$
<code>exists x : S P</code>	$\exists x : S \bullet P$
<code>exists_1 x : S P</code>	$\exists_1 x : S \bullet P$
<code>lambda x : S E</code>	$\lambda x : S \bullet E$
<code>mu x : S P</code>	$\mu x : S \bullet P$

Note: Use `land`, `lor`, `lnot`. English `and/or/not` are not supported.

Sets & Types

<code>x elem S</code>	$x \in S$
<code>x notin S</code>	$x \notin S$
<code>A subset B</code>	$A \subseteq B$
<code>A union B</code>	$A \cup B$
<code>A inter B</code>	$A \cap B$
<code>A setminus B</code>	$A \setminus B$
<code>power S</code>	$\mathbb{P} S$
<code>F S</code>	$\mathbb{F} S$
<code>{x : S P}</code>	Set comprehension
<code>A cross B</code>	$A \times B$
<code>N / Z / N1</code>	$\mathbb{N} / \mathbb{Z} / \mathbb{N}_1$
<code>n..m</code>	$n \dots m$

Z Notation Blocks

<code>given A, B</code>	Given types: $[A, B]$
<code>T ::= a b</code>	Free type
<code>name == expr</code>	Abbreviation
<code>axdef / schema Name / gendef [X]</code>	
<code>declarations</code>	Variable declarations
<code>where</code>	Separator
<code>predicates</code>	Constraining predicates
<code>end</code>	Close the block

Usage

<code>txt2tex f.txt</code>	$\rightarrow$ f.tex + f.pdf
<code>txt2tex f.txt --tex-only</code>	$\rightarrow$ f.tex only
<code>txt2tex -i</code>	Interactive REPL
<code>txt2tex --check-env</code>	Verify dependencies

txt2tex DAT — By Example

Database Design: primary keys, relational algebra, bindings, GROUP/UNGROUP.

Primary Keys

Mark a primary-key attribute with **pk** in a named schema body.

Fuzz-compatible.

You write:

```
schema Class
  pk class : ClassName
  country : CountryName
  bore    : N
end
```

You get:

$Class$

$class : ClassName$
 $country : CountryName$
 $bore : \mathbb{N}$

$PK(Class) = \{class\}$

Composite key: two **pk** lines $\Rightarrow PK(S) = \{a, b\}$. Comma-separated names on one **pk** line also work. **pk** is only valid inside a named **schema**; not allowed in **axdef**, **gendef**, or inline schema text.

Relational Algebra

$\sigma[p](R)$	$\text{Restrict}_p(R)$ — restrict
$\pi[A,B](R)$	$\text{Project}\{A, B\}(R)$ — project
$\rho[A \text{ as } B](R)$	$\text{Rename}_{A \rightarrow B}(R)$ — rename
$R \bowtie S$	$R \otimes S$ — natural join
$R \bowtie [p] S$	$\text{Join}_p(R, S)$ — theta-join
$R \div S$	$R \text{ div } S$ — division

Naming a query. Use **Name == expr** for any right-hand side — pure Z or DAT algebra.

Fuzz-compatible: write algebra as free-standing lines in the document, not inside **zed/axdef/schema** blocks. Schemas and **axdefs** in the same document still type-check.

Long expressions: break at any operator — bare newline or trailing $\backslash$. Break *before* **setminus**, not after.

You write:

```
BigGuns == pi[class,country](
  sigma[bore >= 16](Class))
```

You get:

$BigGuns \hat{=}$ $\text{Project}\{class, country\}(\text{Restrict}_{bore \geq 16}(Class))$

Z Bindings & Set Comprehension

Z RM §3.7 binding brackets construct labelled tuples.

$\{ \text{ name} == e \}$	$\langle name == e \rangle$
$\{ a == e, b == f \}$	multiple components
$\{ \}$	empty binding

Multi-typed comprehension with binding:

You write:

```
{ s : Ship; c : Class |
  s.class = c.class .
  { | name == s.name | } }
```

You get:

$\{ s : Ship; c : Class \mid s.class = c.class \bullet \langle name == s.name \rangle \}$

Use **;** to separate variable-type pairs inside $\{ \}$. Binding components are **comma**-separated (Z RM §3.7).

Fuzz-compatible: write bindings as free-standing expressions in the document, not inside **zed/axdef/schema** blocks.

GROUP & UNGROUP

Date's nested-relation operators.

$R \text{ group } (\{A,B\} \text{ as alias})$	$R \text{ GROUP}(\{A, B\} \text{ AS } alias)$
$R \text{ ungroup alias}$	$R \text{ UNGROUP } alias$

You write:

```
Members group ({name,email} as contact)
Members ungroup contact
```

You get:

$Members \text{ GROUP } (\{name, email\} \text{ AS } contact)$
 $Members \text{ UNGROUP } contact$

Fuzz-compatible: write GROUP/UNGROUP as free-standing expressions in the document, not inside **zed/axdef/schema** blocks.

DAT Quick Reference

pk a : T	primary key in schema
pk a, b : T	composite pk
sigma [p] (R)	restrict
pi [A] (R)	project
rho [A as B] (R)	rename attr
R bowtie S	natural join ($\otimes$)
R div S	division
$\{ a == e \}$	binding bracket
$\{ S; C \mid P . E \}$	multi-typed set comp
R group ({A} as x)	group attrs
R ungroup x	ungroup attr

Identifier Decoration

count'	$count'$ (after-state)
in?	$in?$ (input)
out!	$out!$ (output)
x?'	$x?'$ (runs may combine)
'sunk'	'sunk' (string literal)

Decoration attaches to any identifier. Reserved words (**theta**, **defs**, **hide**, **project**, **Delta**, **Xi**) cannot be decorated.